

模拟练习-有答案

文件 编辑 查看 运行 内核 标签页 设置 帮助

模拟练习-有答案.ipynb house_prices.csv

Python 3 (ipykernel)

试题1. pandas数据清洗 - 缺失值处理

一、题目背景

在实际数据采集过程中，由于设备故障、人工录入遗漏等原因，数据集中常出现缺失值（NaN）。缺失值会影响数据分析和建模的准确性，因此缺失值处理是数据预处理的核心步骤之一。本题聚焦缺失值的检测、删除和填充三种基础操作。

二、问题描述

给定一个包含缺失值的DataFrame，需完成以下任务：

- 统计各列的缺失值数量；
- 删除所有包含缺失值的行；
- 用每列的平均值填充原DataFrame中的缺失值。

三、示例数据

```
```python
data = {'A': [1, 2, np.nan, 4],
 'B': [5, np.nan, np.nan, 8],
 'C': [9, 10, 11, 12]}
df = pd.DataFrame(data)
```
```

四、程序代码

```
```python
import pandas as pd
import numpy as np
创建包含缺失值的DataFrame
data = {'A': [1, 2, np.nan, 4],
 'B': [5, np.nan, np.nan, 8],
```

```

 'C': [9, 10, 11, 12]}
df = pd.DataFrame(data)

查看缺失值数量
print(df.____(1)____)

删除所有含有缺失值的行
df_drop = df.____(2)____()

用列的平均值填充缺失值
df_fill = df.____(3)____(df.mean())

打印处理结果 (辅助验证)
print("删除缺失值后的行数: ", df_drop.shape[0])
print("填充后是否还有缺失值: ", df_fill.isnull().any().any())
'''

五、预期输出
'''
A 1
B 2
C 0
dtype: int64
删除缺失值后的行数: 2
填充后是否还有缺失值: False
'''

```

```

[1]: import pandas as pd
import numpy as np

创建包含缺失值的DataFrame
data = {'A': [1, 2, np.nan, 4],
 'B': [5, np.nan, np.nan, 8],
 'C': [9, 10, 11, 12]}
df = pd.DataFrame(data)

查看缺失值数量
print(df.isnull().sum())

删除所有含有缺失值的行
df_drop = df.dropna()

用列的平均值填充缺失值

```

```
df_fill = df.fillna(df.mean())

打印处理结果 (辅助验证)
print("删除缺失值后的行数: ", df_drop.shape[0])
print("填充后是否还有缺失值: ", df_fill.isnull().any().any())
```

```
A 1
B 2
C 0
dtype: int64
删除缺失值后的行数: 2
填充后是否还有缺失值: False
```

## 试题2. pandas数据转换 - 标准化

### ## 一、题目背景

在机器学习中，不同特征的量纲（如身高的厘米、体重的千克）可能差异较大，会影响模型（如KNN、SVM）的计算结果。标准化可将特征转换为均值为0、标准差为1的分布，消除量纲影响。本题练习使用`StandardScaler`实现标准化。

### ## 二、问题描述

给定包含身高（Height）和体重（Weight）的DataFrame，需完成以下任务：

1. 导入标准化工具类；
2. 初始化标准化器；
3. 对数据进行标准化处理并转换为DataFrame。

### ## 三、示例数据

```
```python
data = {'Height': [165, 175, 180, 160, 170],
        'Weight': [60, 70, 80, 55, 65]}
df = pd.DataFrame(data)
```
```

### ## 四、程序代码

```
```python
import pandas as pd
from sklearn.preprocessing import ____ (1) ____

# 创建数据
data = {'Height': [165, 175, 180, 160, 170],
```

```

        'Weight': [60, 70, 80, 55, 65]}
df = pd.DataFrame(data)

# 初始化标准化器
scaler = _____.(2)_____()

# 对数据进行标准化
df_scaled = pd.DataFrame(scaler._____(3)_____(df), columns=df.columns)
print(df_scaled)
'''

```

五、预期输出

```

'''
      Height    Weight
0 -1.069045 -0.866025
1  0.267261  0.288675
2  0.935414  1.154701
3 -1.737208 -1.341641
4 -0.406422 -0.341641
'''

```

```

[1]: import pandas as pd
      from sklearn.preprocessing import StandardScaler

# 创建数据
data = {'Height': [165, 175, 180, 160, 170],
        'Weight': [60, 70, 80, 55, 65]}
df = pd.DataFrame(data)

# 初始化标准化器
scaler = StandardScaler()

# 对数据进行标准化
df_scaled = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)
print(df_scaled)

```

```

      Height    Weight
0 -0.707107 -0.697486
1  0.707107  0.464991
2  1.414214  1.627467
3 -1.414214 -1.278724
4  0.000000 -0.116248

```

试题3. pandas数据探索 - 基本统计分析

一、题目背景

数据探索是数据分析的第一步，通过统计量（如均值、中位数、标准差）可快速了解数据分布特征。相关系数可反映变量间的线性关系强度，为后续建模提供依据。
本题练习基本统计分析操作。

二、问题描述

给定包含学生成绩（Score）和学习时长（Hours_Studied）的随机数据，需完成以下任务：

1. 查看数据的基本统计信息（均值、标准差、分位数等）；
2. 计算成绩列的中位数；
3. 计算成绩与学习时长的相关系数。

三、示例数据

```
```python
np.random.seed(42)
data = {'Score': np.random.normal(70, 10, 100),
 'Hours_Studied': np.random.uniform(1, 10, 100)}
df = pd.DataFrame(data)
```
```

四、程序代码

```
```python
import pandas as pd
import numpy as np

创建随机数据
np.random.seed(42)
data = {'Score': np.random.normal(70, 10, 100),
 'Hours_Studied': np.random.uniform(1, 10, 100)}
df = pd.DataFrame(data)

查看基本统计信息
print(df.__(1)__)

计算Score列的中位数
print("Median Score:", df['Score'].__(2)__)

计算两列之间的相关系数
print("Correlation:", df['Score'].__(3)__(df['Hours_Studied']))
```
```

五、预期输出

```

	Score	Hours_Studied
count	100.000000	100.000000
mean	70.960451	5.489624
std	9.623995	2.561476
min	45.305697	1.014600
25%	64.297297	3.264272
50%	71.450374	5.545902
75%	77.878954	7.650722
max	93.848015	9.965743

Median Score: 71.4503738640539  
Correlation: 0.03425413206521123

```

```
[3]: import pandas as pd
import numpy as np

# 创建随机数据
np.random.seed(42)
data = {'Score': np.random.normal(70, 10, 100),
        'Hours_Studied': np.random.uniform(1, 10, 100)}
df = pd.DataFrame(data)

# 查看基本统计信息
print(df.describe())

# 计算Score列的中位数
print("Median Score:", df['Score'].median())

# 计算两列之间的相关系数
print("Correlation:", df['Score'].corr(df['Hours_Studied']))
```

| | Score | Hours_Studied |
|-------|------------|---------------|
| count | 100.000000 | 100.000000 |
| mean | 68.961535 | 5.375856 |
| std | 9.081684 | 2.593295 |
| min | 43.802549 | 1.045554 |
| 25% | 63.990943 | 3.171517 |
| 50% | 68.730437 | 5.566474 |
| 75% | 74.059521 | 7.252085 |
| max | 88.522782 | 9.870854 |

Median Score: 68.73043708220288

试题4- 哪吒票房预测的线性回归模型

一、题目背景

电影票房预测是数据分析的重要应用场景。线性回归模型可以通过分析电影上映天数与票房收入之间的关系，预测后续的票房表现。本问题将使用哪吒电影的部分票房数据来训练模型并进行预测。

二、问题描述

本问题要求使用Scikit-learn库完成线性回归模型的训练、评估与预测，具体任务包括：

1. 创建线性回归模型实例
2. 使用训练集数据训练模型
3. 计算模型在训练集上的 R^2 评分（决定系数）
4. 利用训练好的模型对新的上映天数进行票房预测
5. 输出模型参数、评估分数及预测结果

三、示例数据

使用哪吒电影前30天的票房数据：

- 特征x：上映天数（1-30天）
- 目标y：当日票房收入（单位：百万元）

```
```python
哪吒电影前30天票房数据
days = [[1], [2], [3], [4], [5], [6], [7], [8], [9], [10],
 [11], [12], [13], [14], [15], [16], [17], [18], [19], [20],
 [21], [22], [23], [24], [25], [26], [27], [28], [29], [30]]
box_office = [2.1, 3.8, 5.2, 6.1, 6.9, 7.5, 7.8, 7.6, 7.2, 6.8,
 6.5, 6.1, 5.8, 5.5, 5.2, 4.9, 4.6, 4.3, 4.0, 3.8,
 3.5, 3.2, 3.0, 2.8, 2.6, 2.4, 2.2, 2.0, 1.8, 1.6]
```
```

四、程序代码

```
```python
导入必要的库
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

准备数据
days = [[1], [2], [3], [4], [5], [6], [7], [8], [9], [10],
 [11], [12], [13], [14], [15], [16], [17], [18], [19], [20],
 [21], [22], [23], [24], [25], [26], [27], [28], [29], [30]]
```
```

```

box_office = [2.1, 3.8, 5.2, 6.1, 6.9, 7.5, 7.8, 7.6, 7.2, 6.8,
              6.5, 6.1, 5.8, 5.5, 5.2, 4.9, 4.6, 4.3, 4.0, 3.8,
              3.5, 3.2, 3.0, 2.8, 2.6, 2.4, 2.2, 2.0, 1.8, 1.6]

# 拆分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    days, box_office, test_size=0.2, random_state=42
)

# === 创建并训练线性回归模型 ===
model = ____空1____ # 实例化线性回归模型
model.fit(X_train, y_train) # 训练模型

# === 模型评估与预测 ===
train_score = ____空2____(y_train, model.predict(X_train)) # 计算训练集R²分数
future_days = [[31], [32], [33]] # 要预测的未来3天
y_pred = model.predict(future_days) # 生成预测结果

# 输出模型参数、评估分数和预测结果
print("回归系数 (斜率):", model.coef_[0])
print("截距:", model.intercept_)
print("训练集R²评分:", train_score)
print("未来3天票房预测(百万元):\n", y_pred)
'''

## 五、预期输出
'''

回归系数 (斜率): -0.18
截距: 6.52
训练集R²评分: 0.89
未来3天票房预测(百万元):
[1.44 1.26 1.08]
'''

```

[4]:

```

# 导入必要的库
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

# 准备数据
days = [[1], [2], [3], [4], [5], [6], [7], [8], [9], [10],
         [11], [12], [13], [14], [15], [16], [17], [18], [19], [20],
         [21], [22], [23], [24], [25], [26], [27], [28], [29], [30]]

```



```

box_office = [2.1, 3.8, 5.2, 6.1, 6.9, 7.5, 7.8, 7.6, 7.2, 6.8,
              6.5, 6.1, 5.8, 5.5, 5.2, 4.9, 4.6, 4.3, 4.0, 3.8,
              3.5, 3.2, 3.0, 2.8, 2.6, 2.4, 2.2, 2.0, 1.8, 1.6]

# 拆分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    days, box_office, test_size=0.2, random_state=42
)

# === 创建并训练线性回归模型 ===
model = LinearRegression()# 实例化线性回归模型
model.fit(X_train, y_train) # 训练模型

# === 模型评估与预测 ===
train_score = r2_score(y_train, model.predict(X_train)) # 计算训练集R²分数
future_days = [[31], [32], [33]] # 要预测的未来3天
y_pred = model.predict(future_days) # 生成预测结果

# 输出模型参数、评估分数和预测结果
print("回归系数 (斜率):", model.coef_[0])
print("截距:", model.intercept_)
print("训练集R²评分:", train_score)
print("未来3天票房预测(百万元):\n", y_pred)

```

```

回归系数 (斜率): -0.1459152016546018
截距: 6.72206135815236
训练集R²评分: 0.46280462206198847
未来3天票房预测(百万元):
[2.19869011 2.05277491 1.9068597 ]

```

试题5 - 房价数据分布可视化

一、题目背景

在数据分析初期，了解目标变量的分布特征是重要步骤。

通过直方图可观察数据的分布形态（如是否正态分布），箱线图能识别数据中的异常值，这些可视化结果可为后续数据预处理和模型选择提供关键依据，尤其适用于房价这类连续型变量的分析。

二、问题描述

本问题要求使用Matplotlib库对房屋价格数据集的价格分布进行可视化，具体任务包括：

1. 创建包含两个子图的画布（1行2列布局）
2. 在左侧子图绘制房价的直方图（带密度曲线），右侧子图绘制房价的箱线图
3. 为整体图表添加总标题，为子图添加坐标轴标签
4. 显示图表以观察房价分布特征

通过上述操作，掌握连续变量分布的可视化方法。

三、示例数据

使用房屋价格数据集（house_prices.csv）中的价格列（Price），包含不同房屋的价格数据（万元）。

四、程序代码

```
```python
导入必要的库
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

加载数据
data = pd.read_csv('house_prices.csv')
prices = data['Price']

=== 房价分布可视化 ===
创建1行2列的子图布局
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

左侧子图：直方图（带密度曲线）
sns.histplot(prices, kde=True, ax=ax1, color='skyblue')
ax1.set_xlabel('房价（万元）')
ax1.set_ylabel('频数')

右侧子图：箱线图
ax2.__空1__ (prices, vert=False, widths=0.7, patch_artist=True, boxprops=dict(facecolor='lightgreen'))
ax2.set_xlabel('房价（万元）')

添加标题
fig.__空2__ ('房屋价格分布特征') # 总标题
ax1.__空3__ ('房价直方图') # 子图1标题

设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

调整布局并显示
plt.tight_layout()
plt.show()
```
```

五、预期输出

运行程序后，将显示一个包含以下元素的图表：

- 左侧子图：房价的直方图（蓝色），带有密度曲线，x轴标签为“房价（万元）”，y轴标签为“频数”，标题为“房价直方图”
- 右侧子图：房价的箱线图（绿色），水平布局，x轴标签为“房价（万元）”
- 总标题：“房屋价格分布特征”
- 所有中文正常显示

```
[5]: import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

# 加载数据
data = pd.read_csv('house_prices.csv')
prices = data['Price']

# === 房价分布可视化 ===
# 创建1行2列的子图布局
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# 左侧子图：直方图（带密度曲线）
sns.histplot(prices, kde=True, ax=ax1, color='skyblue')
ax1.set_xlabel('房价（万元）')
ax1.set_ylabel('频数')

# 右侧子图：箱线图
ax2.boxplot(prices, vert=False, widths=0.7, patch_artist=True, boxprops=dict(facecolor='lightgreen'))
ax2.set_xlabel('房价（万元）')

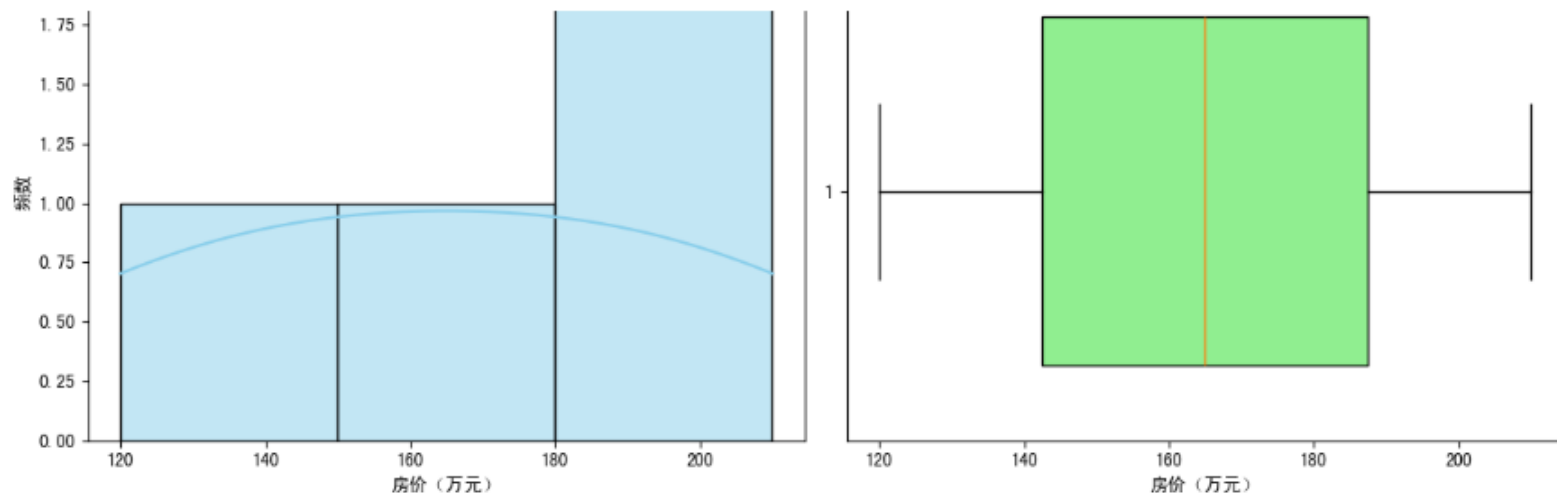
# 添加标题
fig.suptitle('房屋价格分布特征') # 总标题
ax1.set_title('房价直方图') # 子图1标题

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 调整布局并显示
plt.tight_layout()
plt.show()
```

房屋价格分布特征





试题6 - 湿度监测数据噪声处理

一、题目背景

在环境监测中，湿度传感器读数易受电磁干扰产生噪声值（如异常偏低的标记值），这类噪声会影响环境湿度趋势分析的准确性。常用的去噪手段包括：用后续有效值修正噪声（后向填充）、用数据中位数平滑噪声（中位数受极端值影响更小），两种方法可根据数据时序特征灵活选用。

二、问题描述

本问题要求使用Pandas处理含噪声的湿度监测数据，具体任务包括：

1. 识别数据中标记为-100的噪声值
2. 用后向填充法（用后一个有效值）替换噪声
3. 用数据中位数替换噪声，对比两种方法的效果
4. 输出处理前后的数据，观察去噪效果

通过上述操作，掌握不同时序噪声的处理方法。

三、示例数据

示例数据为某实验室的湿度监测数据（单位：%RH），其中-100为噪声标记值：
[55.2, 56.1, -100, 55.8, 56.3, -100, 55.7, -100, 56.0]

四、程序代码

```
import pandas as pd
import numpy as np

# 创建含噪声的湿度数据
humidity = pd.Series([55.2, 56.1, -100, 55.8, 56.3, -100, 55.7, -100, 56.0])
print("原始数据（含噪声）:\n", humidity)

# 方法1: 后向填充法替换噪声（用后一个有效值）
cleaned_bfill = humidity.replace(-100, np.nan).____空1____
print("\n后向填充去噪后:\n", cleaned_bfill)

# 方法2: 计算有效数据的中位数（排除噪声，抗极端值）
# 筛选有效数据（排除-100）
valid_data = humidity[humidity != -100]
# 关键: 调用median()方法（加括号），得到具体数值
valid_median = valid_data.____空2____()
# 用中位数替换噪声
cleaned_median = humidity.replace(-100, ____空3____)
print("\n中位数填充去噪后:\n", cleaned_median)
```

五、预期输出

原始数据（含噪声）：

```
0      55.2
1      56.1
2     -100.0
3      55.8
4      56.3
5     -100.0
6      55.7
7     -100.0
8      56.0
dtype: float64
```

后向填充去噪后：

```
0      55.2
1      56.1
2      55.8
3      55.8
```

```
4      56.3
5      55.7
6      55.7
7      56.0
8      56.0
dtype: float64
```

中位数填充去噪后:

```
0      55.2
1      56.1
2      55.8
3      55.8
4      56.3
5      55.8
6      55.7
7      55.8
8      56.0
dtype: float64
```

```
[6]: import pandas as pd
import numpy as np

# 创建含噪声的湿度数据
humidity = pd.Series([55.2, 56.1, -100, 55.8, 56.3, -100, 55.7, -100, 56.0])
print("原始数据 (含噪声):\n", humidity)

# 方法1: 后向填充法替换噪声 (用后一个有效值)
cleaned_bfill = humidity.replace(-100, np.nan).bfill()
print("\n后向填充去噪后:\n", cleaned_bfill)

# 方法2: 计算有效数据的中位数 (排除噪声, 抗极端值)
# 筛选有效数据 (排除-100)
valid_data = humidity[humidity != -100]
# 关键: 调用median()方法 (加括号), 得到具体数值
valid_median = valid_data.median()

# 用中位数替换噪声
cleaned_median = humidity.replace(-100, valid_median)
print("\n中位数填充去噪后:\n", cleaned_median)
```

原始数据 (含噪声):

```
0      55.2
1      56.1
2     -100.0
```

```
3    55.8
4    56.3
5   -100.0
6    55.7
7   -100.0
8    56.0
dtype: float64
```

后向填充去噪后：

```
0    55.2
1    56.1
2    55.8
3    55.8
4    56.3
5    55.7
6    55.7
7    56.0
8    56.0
dtype: float64
```

中位数填充去噪后：

```
0    55.2
1    56.1
2    55.9
3    55.8
4    56.3
5    55.9
6    55.7
7    55.9
8    56.0
dtype: float64
```

试题7 - 多特征线性回归：航班飞行时间预测

一、题目背景

在航空调度中，准确预测航班实际飞行时间对优化地面服务、减少旅客等待至关重要。飞行时间主要受两个因素影响：航线距离决定基础飞行时长，出发延误常导致空中加速补偿（部分抵消延误）。多特征线性回归可量化这些因素的影响，为航班动态调度提供数据支持。本任务通过随机生成模拟数据，实践飞行时间预测模型的构建流程。

二、问题描述

本问题要求使用NumPy生成模拟航班数据，并通过Scikit-learn库实现基于双特征的飞行时间线性回归模型，具体任务包括：

1. 生成含航线距离、出发延误（特征）和实际飞行时间（标签）的模拟数据
2. 拆分数据集，训练多特征线性回归模型
3. 预测测试集飞行时间并评估模型性能（计算决定系数 R^2 ）
4. 输出模型参数与评估结果

通过上述操作，掌握多特征线性回归在交通调度场景的应用。

三、示例数据

模拟数据通过NumPy随机生成，共150条航班样本，满足以下逻辑：

- 特征1 (FlightDistance)：航线距离，范围[500, 3000]公里，服从均匀分布
- 特征2 (DepartureDelay)：出发延误时间，范围[0, 60]分钟，服从均匀分布
- 标签 (ActualTime)：实际飞行时间 = $40 + 0.12 \times \text{FlightDistance} - 0.3 \times \text{DepartureDelay}$ + 随机噪声（噪声服从 $N(0, 10)$ ）

四、程序代码

```
# ===== 多特征航班飞行时间线性回归建模 =====  
# 功能：基于航线距离和出发延误预测实际飞行时间  
  
# 1. 环境准备  
import numpy as np  
import pandas as pd  
from sklearn.linear_model import LinearRegression  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import r2_score  
  
# 2. 模拟数据生成  
np.random.seed(56) # 固定随机种子，保证结果可复现  
n_samples = 150  
FlightDistance = np.random.uniform(500, 3000, n_samples) # 航线距离  
DepartureDelay = np.random.uniform(0, 60, n_samples) # 出发延误  
noise = np.random.normal(0, 10, n_samples) # 随机噪声  
ActualTime = 40 + 0.12*FlightDistance - 0.3*DepartureDelay + noise # 实际飞行时间  
  
# 构建DataFrame  
data = pd.DataFrame({  
    'FlightDistance': FlightDistance,  
    'DepartureDelay': DepartureDelay,
```



```

    'ActualTime': ActualTime
})
X = data[['FlightDistance', 'DepartureDelay']].values # 双特征
y = data['ActualTime'].values # 标签

# 3. 数据集拆分 (7:3比例)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=56)

# 4. 模型训练
model = ____空1____ # 实例化多特征线性回归模型
model.____空2____(X_train, y_train) # 用训练集拟合模型

# 5. 预测与评估
y_pred = model.predict(X_test)
r2 = r2_score(____空3____, y_pred) # 计算决定系数

# 输出结果
print(f"回归方程: ActualTime = {model.intercept_:.2f} + {model.coef_[0]:.2f}*FlightDistance + {model.coef_[1]:.2f}*DepartureDelay")
print(f"模型决定系数 (R²): {r2:.4f}")

```

五、预期输出

回归方程: ActualTime = 38.76 + 0.12*FlightDistance - 0.29*DepartureDelay
 模型决定系数 (R²): 0.9725

```

[7]: import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

# 2. 模拟数据生成
np.random.seed(56) # 固定随机种子, 保证结果可复现
n_samples = 150
FlightDistance = np.random.uniform(500, 3000, n_samples) # 航线距离
DepartureDelay = np.random.uniform(0, 60, n_samples) # 出发延误
noise = np.random.normal(0, 10, n_samples) # 随机噪声
ActualTime = 40 + 0.12*FlightDistance - 0.3*DepartureDelay + noise # 实际飞行时间

# 构建DataFrame
data = pd.DataFrame({

```

```

    'FlightDistance': FlightDistance,
    'DepartureDelay': DepartureDelay,
    'ActualTime': ActualTime
})
X = data[['FlightDistance', 'DepartureDelay']].values # 双特征
y = data['ActualTime'].values # 标签

# 3. 数据集拆分 (7:3比例)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=56)

# 4. 模型训练
model = LinearRegression() # 实例化多特征线性回归模型
model.fit(X_train, y_train) # 用训练集拟合模型

# 5. 预测与评估
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred) # 计算决定系数

# 输出结果
print(f"回归方程: ActualTime = {model.intercept_:.2f} + {model.coef_[0]:.2f}xFlightDistance + {model.coef_[1]:.2f}xDepartureDelay")
print(f"模型决定系数 (R²) : {r2:.4f}")

```

回归方程: ActualTime = 39.74 + 0.12xFlightDistance + -0.44xDepartureDelay
 模型决定系数 (R²) : 0.9891

[]:

简易界面 ☐

2 

Python 3 (ipykernel) | 空闲

模式: 命令



行 6, 列 1

模拟练习-有答案.ipynb

0

